

Core Exam Practice

COMP 352 – Data Structures and Algorithms

AI-generated exam-style practice questions for study purposes only. Not official course material. Original practice exercises with worked answers; no real exam questions are reproduced.

Based on your supplied outline: COMP 352 Course Outline, Winter 2025, in comp352.pdf. The course description and reading list appear on printed page 2 (PDF page 3); exam formats and the tentative topic schedule appear on printed page 4 (PDF page 5). This edition uses that outline as its scope reference, not as confirmation of a current offering.

How to use this set

Pages 2–7: attempt the questions before reading solutions. Page 8: objective-question answers. Pages 9–13: worked solutions and marking guidance. Use the ruled spaces or separate paper. Suggested practice time: 150–180 minutes, or split into topic sessions. This timing is a study suggestion.

120 practice marks: 8 multiple-choice questions x 1 mark; 6 true/false questions x 2 marks; 10 structured questions x 10 marks. For true/false, earn 1 mark for the decision and 1 for a valid justification. No negative marking is used here; the supplied outline mentions it for course multiple-choice exams but does not specify a rate.

Outline-to-practice map

Algorithm analysis and recursion: Q1–Q2 (page 3).

Arrays, stacks, queues and lists: Q3–Q4 (page 4).

Trees, search trees, priority queues and heaps: Q5–Q6 (page 5).

Maps, hash tables and sorting: Q7–Q8 (page 6).

Graphs and data-structure design: Q9–Q10 (page 7).

This is a core practice selection, not exhaustive coverage of every listed textbook section. Graph practice focuses on representations, BFS and DFS. Topic ordering, scope and assessment details may differ in your current course.

EazyGrades is not affiliated with, endorsed by, or officially connected to Concordia University or any school. No grade or exam outcome is guaranteed. Use for independent revision; do not submit these answers as assessed coursework.

Quick concept check

- M1.** Which is a tight bound for $5n^2 + 3n + 8$?
A $O(n)$ B $\Theta(n^2)$ C $\Theta(n^3)$ D $\Theta(\log n)$
- M2.** Which structure naturally implements recursive-call bookkeeping?
A queue B hash table C stack D sorted array
- M3.** Worst-case access to index i in a singly linked list of n nodes?
A $\Theta(1)$ B $\Theta(\log n)$ C $\Theta(n)$ D $\Theta(n \log n)$
- M4.** In a zero-based binary heap, the left child of index i is at:
A $2i$ B $2i+1$ C $i/2$ D $2i+2$
- M5.** Which traversal of a BST with distinct keys produces sorted order?
A preorder B postorder C level-order D inorder
- M6.** Which structure is central to standard BFS?
A FIFO queue B LIFO stack C min-heap D BST
- M7.** Typical array mergesort worst-case time is:
A $\Theta(n)$ B $\Theta(n^2)$ C $\Theta(n \log n)$ D $\Theta(2^n)$
- M8.** Two unequal keys hashing to one index is called:
A recursion B collision C traversal D rotation

True or false – justify each answer

- T1.** Every binary search tree has logarithmic height.
- T2.** An array-based stack can have amortized $O(1)$ push with occasional linear work.
- T3.** A min-heap array must be globally sorted.
- T4.** Binary search works correctly on any unsorted array.
- T5.** A hash-table lookup is always worst-case $O(1)$.
- T6.** BFS gives minimum-edge-count paths from its source in an unweighted graph.

Write each decision and a one-sentence explanation on separate paper.

Analysis and recursion

Q1. Algorithm analysis / 10 marks

- (a) Let n be a positive power of two. An outer loop takes $i = 1, 2, 4, \dots, n$. At each i , an inner loop executes exactly i times. Give the exact total inner executions and a tight asymptotic bound. (4 marks)
- (b) Order these functions by increasing asymptotic growth: $n \log_2(n)$, 2^n , $\log_2(n)$, n^2 , n . (3 marks)
- (c) Is $3n + 7$ in $O(n^2)$? Is it in $\Theta(n^2)$? Explain the distinction. (3 marks)

Q2. Recursion and correctness / 10 marks

- (a) For powers of two, $T(1) = 1$ and $T(n) = 2T(n/2) + n$. Give the exact solution and explain the work at each recursion level. (4 marks)
- (b) Recursive binary search uses $\text{mid} = \text{floor}((\text{lo} + \text{hi})/2)$. On a sorted array, it recurses into $[\text{lo}, \text{mid}-1]$ or $[\text{mid}+1, \text{hi}]$. State the empty-range base case and explain termination. (3 marks)
- (c) State binary search worst-case time and recursive auxiliary space. How does an iterative version change the space bound? (3 marks)

Linear data structures

Q3. Arrays, stacks and queues / 10 marks

(a) A circular queue has capacity 5, front = 3 and size = 3. Its logical contents are A, B, C in that order. Show physical indices. Dequeue once, then enqueue D and E; give final front, size and array slots. (4 marks)

(b) Starting empty, execute push(4), push(7), pop(), push(2), push(9), pop(). List returned values and final stack, bottom to top. (3 marks)

(c) An array doubles capacity when full. Explain why append is amortized $O(1)$ even though one append can take $\Theta(n)$. (3 marks)

Q4. Linked lists and implementation / 10 marks

(a) Write Java-like pseudocode to reverse a singly linked list in place. Preserve every node, return the new head, and handle an empty list. (5 marks)

(b) Give tight time and auxiliary-space bounds for your reversal. (2 marks)

(c) With head and tail pointers, compare removing the last node of a singly linked list and a doubly linked list. Explain. (3 marks)

Trees and priority queues

Q5. Trees, traversals and search / 10 marks

- (a) Insert 7, 3, 9, 1, 5 into an empty binary search tree. Draw the tree, then give inorder and preorder traversals. (4 marks)
- (b) Delete 7 using its inorder successor as the replacement. Draw the resulting tree and state the successor. (3 marks)
- (c) Define height as the number of edges on the longest root-to-leaf path. Give the original height and search cost in terms of height; compare balanced and degenerate trees. (3 marks)

Q6. Priority queues and heaps / 10 marks

- (a) Insert 6, 2, 9, 1, 5 into an empty binary min-heap. Show the level-order array after each insertion. Use zero-based indexing. (4 marks)
- (b) Perform removeMin on the final heap. Show the replacement and down-heap swaps, then the final array. (3 marks)
- (c) Give the costs of min(), insert(), removeMin(), and bottom-up construction from n keys. (3 marks)

Hashing and sorting

Q7. Maps and hash tables / 10 marks

(a) Insert keys 10, 17, 24, 5 into a table of size 7 using $h(k) = k \bmod 7$ and linear probing. Show occupied indices and the load factor. (4 marks)

(b) Delete key 17. Why should its slot be marked with a tombstone rather than made never-used? Trace a search for 24. (3 marks)

(c) Contrast expected lookup under well-distributed hashes and controlled load with worst-case lookup. What can cause the worst case? (3 marks)

Q8. Sorting and algorithm choice / 10 marks

(a) Trace insertion sort on [4,1,3,2], giving the whole array after each outer-loop pass. State worst-case time and auxiliary space. (4 marks)

(b) Merge sorted runs [2,6,8] and [1,3,7]. Give the result and number of key comparisons; stop comparing when one run is exhausted. (3 marks)

(c) Which is preferable for a nearly sorted small array: insertion sort or a typical mergesort implementation? Explain, and define stability for equal-key records. (3 marks)

Graphs and design

Q9. Graph representation and traversal / 10 marks

Use an undirected graph with vertices A, B, C, D, E, F and edges AB, AC, BD, CD, DE. Visit neighbors alphabetically. (a) Give adjacency lists and BFS discovery order from A, including distances and parents. (5 marks)

(b) Give recursive DFS discovery order from A. Is F reachable? (2 marks)

(c) Give a shortest A-to-E path. Explain why BFS finds shortest paths in unweighted graphs, and state its adjacency-list time bound. (3 marks)

Q10. Choosing structures and designing operations / 10 marks

A task scheduler stores unique task IDs with integer priorities. Smaller numbers run first; tied priorities may run in either order. It supports add, lookup by ID, and remove-next. (a) Propose a combination of data structures and explain what each stores. (4 marks)

(b) State the expected cost of each operation and describe how you keep the structures consistent when removing the next task. (3 marks)

(c) If priority changes by ID must take $O(\log n)$ expected time, what extra information must you maintain? Explain how updates preserve it. (3 marks)

SECTION A / ANSWER KEY

Concept-check solutions

Multiple choice / 8 marks

- M1.** B: the quadratic term dominates.
- M2.** C: the most recent unfinished call resumes first.
- M3.** C: reaching a late position requires following links.
- M4.** B: left child is $2i+1$; right child is $2i+2$.
- M5.** D: left subtree, node, right subtree.
- M6.** A: FIFO processing preserves distance layers.
- M7.** C: linear merging across logarithmically many levels.
- M8.** B: collisions require a resolution strategy.

True or false / 12 marks

- T1.** False. Sorted insertions into an unbalanced BST can produce height $n-1$.
- T2.** True. Geometric capacity growth spreads total copying over many pushes.
- T3.** False. Parent-child order does not order siblings or unrelated subtrees.
- T4.** False. Its half-discarding argument depends on sorted order.
- T5.** False. Collisions can force a linear probe sequence or long bucket.
- T6.** True. Vertices are first discovered in nondecreasing distance layers.

For each true/false item, award one mark for the decision and one for a correct explanation. Accept equivalent counterexamples or arguments.

Worked solutions 1–2

Q1. Algorithm analysis

- (a) The work is $1 + 2 + \dots + n = 2n - 1$, hence $\Theta(n)$. There are $\log_2(n) + 1$ outer iterations, but their costs are not all n . Award 2 marks for the sum, 1 for its value, 1 for the bound.
- (b) $\log_2(n)$, n , $n \log_2(n)$, n^2 , 2^n . This is an eventual-growth comparison, not a claim about every small n . Award 3 marks for the full ordering; 2 for one adjacent inversion.
- (c) Yes to $O(n^2)$: for $n \geq 1$, $3n + 7 \leq 10n^2$. No to $\Theta(n^2)$, since $(3n + 7)/n^2$ tends to 0. Its tight growth is $\Theta(n)$. Award one mark for each conclusion and one for reasoning.

Q2. Recursion and correctness

- (a) There are $\log_2(n)$ internal levels, each doing n nonrecursive work. The n leaves contribute n , so $T(n) = n \log_2(n) + n = \Theta(n \log n)$. Award 2 marks for level work/depth, 1 for leaves, 1 for the result.
- (b) If $lo > hi$, return not found. Otherwise either the midpoint matches or the next interval is strictly smaller, because mid is excluded. A nonnegative interval size cannot decrease forever. Award one mark each for base case, shrinking range and termination argument.
- (c) Worst-case time is $\Theta(\log n)$, with $\Theta(\log n)$ recursive stack space. Iteration uses $\Theta(1)$ auxiliary space. Award one mark each. Assume constant-time array access.

Self-review: identify whether each missed mark came from a trace, a boundary case, a complexity assumption, or an explanation. Retry the question with different inputs before reviewing the solution again.

Worked solutions 3–4

Q3. Arrays, stacks and queues

(a) Initially A is at 3, B at 4, C at 0. Removing A gives front = 4, size = 2. D goes to $(4+2) \bmod 5 = 1$; E goes to $(4+3) \bmod 5 = 2$. Final front = 4, size = 4, slots 0..4 = [C, D, E, unused, B]. Award one mark per stage; stale removed data in slot 3 is acceptable if labelled inactive.

(b) The pops return 7 and 9. The final stack is [4, 2] bottom to top. Award 1 mark for each pop and 1 for the stack.

(c) With initial capacity 1, resizing copies $1 + 2 + 4 + \dots$ items, totaling less than $2n$ over n appends. Including n new writes gives $O(n)$ total work and $O(1)$ amortized cost. A resizing append copies all current items. Award one mark for the series, one for total/amortized cost, one for the single-operation caveat.

Q4. Linked lists and implementation

(a) Initialize prev = null and curr = head. While curr != null: save next = curr.next; set curr.next = prev; set prev = curr; set curr = next. Return prev. Saving next before changing the link prevents losing the unprocessed suffix. Empty input returns null. Award 1 mark each for initialization, saved next, link reversal, pointer progress, and return/empty handling.

(b) Each node is processed once: $\Theta(n)$ time and $\Theta(1)$ auxiliary space. Award one mark each.

(c) A singly linked list needs $\Theta(n)$ worst-case time to locate the predecessor of tail; tail alone cannot identify it. A doubly linked list uses tail.prev in $\Theta(1)$ time. Handle the singleton case by clearing head and tail, otherwise clear the new tail.next. Award one mark per complexity and one for explanation/boundaries.

Self-review: identify whether each missed mark came from a trace, a boundary case, a complexity assumption, or an explanation. Retry the question with different inputs before reviewing the solution again.

Worked solutions 5–6

Q5. Trees, traversals and search

(a) Root 7 has left child 3 and right child 9; node 3 has children 1 and 5. Inorder: 1, 3, 5, 7, 9. Preorder: 7, 3, 1, 5, 9. Award 2 marks for the tree and 1 per traversal.

(b) The minimum node in the right subtree is 9, the successor. Replace 7 with 9 and remove its original node. Root 9 now has only left child 3, whose children remain 1 and 5. Award one mark for successor and two for the resulting structure.

(c) Original height $h = 2$. Worst-case search is $O(h+1)$, commonly written $O(h)$ for positive heights. Balanced trees give $O(\log n)$; a degenerate tree gives $O(n)$. Award one mark for height, one for height-based work and one for the comparison.

Q6. Priority queues and heaps

(a) Arrays: [6]; [2,6]; [2,6,9]; [1,2,9,6]; [1,2,9,6,5]. The parent of index $i > 0$ is $\text{floor}((i-1)/2)$. Up-heap swaps restore parent $\leq$ child. Award 1 mark for each of the final four arrays.

(b) Return 1. Move the last key 5 to root: [5,2,9,6]. Swap with smaller child 2, yielding [2,5,9,6]. Now $5 \leq 6$, so stop. Award one mark each for returned/replacement values, chosen swap and final heap.

(c) $\text{min}()$ is $\Theta(1)$. Insert and removeMin are $O(\log n)$ worst case; bottom-up construction is $\Theta(n)$. The latter is not n full-height insertions: most nodes have small subtree height. Award 1 mark for min , 1 for updates, 1 for construction. Assume enough capacity so array resizing is excluded.

Self-review: identify whether each missed mark came from a trace, a boundary case, a complexity assumption, or an explanation. Retry the question with different inputs before reviewing the solution again.

Worked solutions 7–8

Q7. Maps and hash tables

(a) 10 goes to 3, 17 probes to 4, 24 probes to 5, and 5 probes to 6. Load factor is $4/7$. Award 3 marks for correct placements and 1 for load factor.

(b) Index 4 becomes a tombstone. Searching for 24 begins at 3, skips 10, passes the tombstone at 4, and finds 24 at 5. A never-used slot would incorrectly terminate the search at 4. Award one mark for deletion marker, trace and explanation.

(c) Expected lookup is $O(1)$ under suitable hashing and a load factor bounded away from 1. Worst-case lookup is $O(n)$ for a table of size $\Theta(n)$, because many collisions can form a long probe cluster. Award one mark per bound and one for cause/assumptions.

Q8. Sorting and algorithm choice

(a) Passes: [1,4,3,2], [1,3,4,2], [1,2,3,4]. Worst-case time is $\Theta(n^2)$, auxiliary space $\Theta(1)$. Award 2 marks for the passes and 1 per bound.

(b) Result: [1,2,3,6,7,8]. Compare (2,1), (2,3), (6,3), (6,7), (8,7): five comparisons, then append 8. Award 1 mark for output and 2 for count/trace.

(c) Insertion sort is often preferable because overhead is small and only a few shifts are needed; this is a practical choice, not a universal performance guarantee. A stable sort preserves original relative order among equal-key records. Insertion sort is stable if it shifts only strictly larger keys. Award one mark for choice, reason and definition.

Self-review: identify whether each missed mark came from a trace, a boundary case, a complexity assumption, or an explanation. Retry the question with different inputs before reviewing the solution again.

Worked solutions 9–10

Q9. Graph representation and traversal

(a) Lists: A: B,C; B: A,D; C: A,D; D: B,C,E; E: D; F: empty. BFS discovers A,B,C,D,E. Distances are 0,1,1,2,3, respectively; F is unreachable. Parents: $B \leftarrow A$, $C \leftarrow A$, $D \leftarrow B$, $E \leftarrow D$. Award 2 marks for lists, 1 for order, 1 for distances and 1 for parents.

(b) DFS discovers A,B,D,C,E. F is not reachable from A. Award one mark each. This is recursive DFS; returning from C to D precedes discovering E.

(c) A-B-D-E has length 3 (A-C-D-E is also shortest). A FIFO queue processes vertices in increasing distance layers, so first discovery supplies a shortest path in edge count. The standard full-graph bound is $O(V+E)$, including initialization. Award one mark each for path, reasoning and bound.

Q10. Choosing structures and designing operations

(a) Use a binary min-heap of task records for priority order and a hash map from ID to the task record (or its heap index). Reject duplicate IDs. Heap keys are priorities, not IDs. Award 2 marks for the heap, 1 for the map and 1 for a consistent ID association.

(b) With suitable hashing, lookup is expected $O(1)$; add and remove-next are expected $O(\log n)$ amortized if dynamic storage may resize. On removal, remove the heap root and delete the same ID from the map; repair the heap. Empty removal returns an explicit empty result. Award one mark for lookup, one for updates and one for synchronized deletion.

(c) Maintain ID-to-heap-index mapping. Find the item in expected $O(1)$, change its priority, then up-heap for a decrease or down-heap for an increase. Every swap updates both moved IDs in the map. At most $O(\log n)$ levels are traversed. Award one mark for index mapping, one for repair direction and one for swap consistency.

Self-review: identify whether each missed mark came from a trace, a boundary case, a complexity assumption, or an explanation. Retry the question with different inputs before reviewing the solution again.